Introducción: ¿Para qué necesitamos UML?

INTEGRANTE 1 — PONENTE PRINCIPAL

Hasta ahora hemos creado varios programas en Java: empleados, fechas, figuras geométricas y un sistema hospitalario. Pero hay algo que no siempre resulta fácil: entender cómo se conecta todo cuando solamente tenemos el código enfrente.

Por eso, en la práctica nueve trabajamos con UML, o Lenguaje de Modelado Unificado, que permite representar programas mediante diagramas.

No se trata de volver a escribirlos, sino de mostrar su estructura y comportamiento.

Retomamos ejercicios de las prácticas cinco, siete y ocho, además de la segunda etapa del proyecto hospitalario.

El trabajo se dividió en dos partes: identificar qué clases forman cada sistema y seguir las acciones de sus objetos.

Diagramas de clases y cardinalidad

INTEGRANTE 2

Primero están los diagramas de clases. ¿Cómo saber qué contiene un programa sin revisar todos sus archivos?

En estos diagramas, cada clase aparece como un recuadro con su nombre, atributos y métodos.

Los atributos son los datos que guarda, mientras que los métodos son las acciones que realiza.

También representamos relaciones como la herencia, cuando una clase obtiene características de otra.

Y añadimos la cardinalidad, que indica cuántos objetos pueden relacionarse.

Por ejemplo, en el ejercicio de figuras geométricas, un cuadrilátero está formado por cuatro puntos.

Así podemos reconocer su estructura de un vistazo.

Diagramas de secuencia

INTEGRANTE 3

Pero conocer las piezas de un programa no nos dice qué ocurre cuando lo usamos.

Para eso elaboramos diagramas de secuencia.

Cada objeto tiene una línea de vida, y las flechas muestran los mensajes que intercambian conforme avanza una operación.

Esas flechas representan llamadas a métodos y sus respuestas.

Por ejemplo, en el ejercicio de empleados podemos seguir cómo se crean los objetos, se consultan sus datos y se modifica un salario.

En el de fechas, observamos el orden en que se reciben, cambian y muestran sus valores.

Así entendemos no solo qué hace el programa, sino cuándo sucede cada acción.

Aplicación en las prácticas 5 y 7

INTEGRANTE 4

Con ambos tipos de diagramas retomamos los programas anteriores.

En la práctica cinco representamos los ejercicios de empleados y fechas, identificando atributos, constructores y métodos.

Para la práctica siete dibujamos la jerarquía de cuadriláteros: partimos de una clase general y seguimos sus relaciones con trapecios, paralelogramos, rectángulos y cuadrados.

Esto nos permitió reconocer la herencia y cómo se organizan las clases.

Después comparamos los diagramas con el código original para comprobar que las relaciones y operaciones dibujadas correspondieran con lo que realmente habíamos programado.

Práctica 8: Interfaces y polimorfismo

INTEGRANTE 5

En la práctica ocho trabajamos con la huella de carbono.

Aquí el caso era diferente: edificios, autos y bicicletas no calculan su huella de la misma manera.

Lo que comparten es la interfaz HuellaCarbono, que establece un método común llamado getHuellaCarbono.

Cada clase lo implementa a su manera.

El diagrama de clases muestra esa relación, mientras que el de secuencia sigue la creación de objetos y sus cálculos.

También aparece el polimorfismo: una misma llamada puede producir comportamientos distintos dependiendo del objeto que la recibe.

Y esa idea vuelve a aparecer en el proyecto hospitalario.

Proyecto hospitalario: Estructura y funcionamiento

INTEGRANTE 1 — PONENTE PRINCIPAL

El sistema hospitalario fue la parte más amplia, porque aquí varias clases necesitan colaborar.

Empezamos revisando su organización: personal médico, pacientes, sus clases derivadas, la interfaz AtencionMedica y la clase Sistema, encargada de administrar registros y asignaciones.

En el diagrama de clases representamos esas relaciones e incluimos restricciones, como el límite de pacientes que puede atender un médico.

Después construimos el diagrama de secuencia siguiendo las acciones principales.

Primero aparece el registro de médicos, enfermeros y pacientes.

Más adelante, cuando queremos asignar un paciente, el sistema busca al médico y al paciente por nombre, y después intenta realizar la asignación.

Algo importante es que no envía la solicitud a todos los médicos: utiliza el objeto correspondiente.

Para las consultas y los tratamientos mostramos las comprobaciones necesarias antes de cambiar el estado del paciente.

También incluimos acciones de enfermería, como asignar pacientes, incorporar integrantes a un equipo y supervisarlo.

Aquí se entiende mejor el polimorfismo.

Mediante AtencionMedica podemos llamar al método de tratamiento, pero la acción concreta depende de si interviene un médico o un enfermero.

El diagrama permite seguir esa comunicación paso a paso, identificando qué objeto se encarga de cada tarea.

Conclusión y resultados

INTEGRANTE 1 — PONENTE PRINCIPAL

Al terminar, reunimos los diagramas y los contrastamos con los programas originales.

Así cumplimos los objetivos: documentar nuestros sistemas con UML, representar clases y cardinalidades, y mostrar el orden de los mensajes entre objetos.

Lo más útil fue descubrir que un programa puede entenderse desde dos perspectivas: cómo está construido y qué sucede cuando funciona.

En un proyecto pequeño eso ya ayuda bastante; en uno tan amplio como el hospitalario, hace mucho más fácil seguir las relaciones e identificar dónde interviene cada clase.